

Convolution.

Blumenau Mark

Why don't fully connected layers solve everything?

Let's take MNIST, 28 x 28 pixels -> 784 numbers at the input

10 numbers at the output (number of digits)

Let's say there are 1000 parameters in the hidden layer, then the total is:

$(784+1)*1000 + (1000 + 1) * 10 = 785,000 + 10,010 = \text{a lot}$

And what does a lot mean? Overfitting might kill us...

But we have Dropout!

Well, it is possible to solve MNIST without it

Table 1: Error rates on MNIST test set.

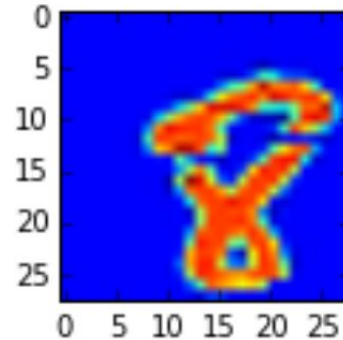
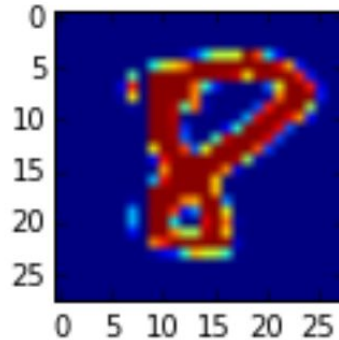
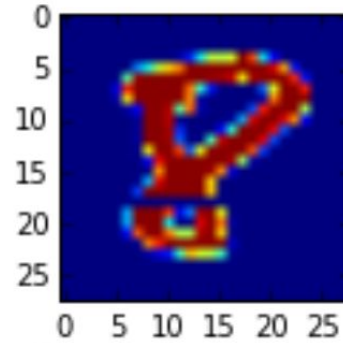
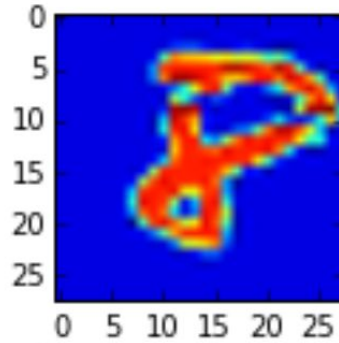
ID	architecture (number of neurons in each layer)	test error for best validation [%]	best test error [%]	simulation time [h]	weights [millions]
1	1000, 500, 10	0.49	0.44	23.4	1.34
2	1500, 1000, 500, 10	0.46	0.40	44.2	3.26
3	2000, 1500, 1000, 500, 10	0.41	0.39	66.7	6.69
4	2500, 2000, 1500, 1000, 500, 10	0.35	0.32	114.5	12.11
5	9×1000 , 10	0.44	0.43	107.7	8.86

And if it doesn't work out, we'll use dropout, right?

Think about it: let's take a 224 x 224 image. And then layers 1000, 500, 100, and 10.

Dropout is not the best solution. The best cure for our problem is to kill the parameters!

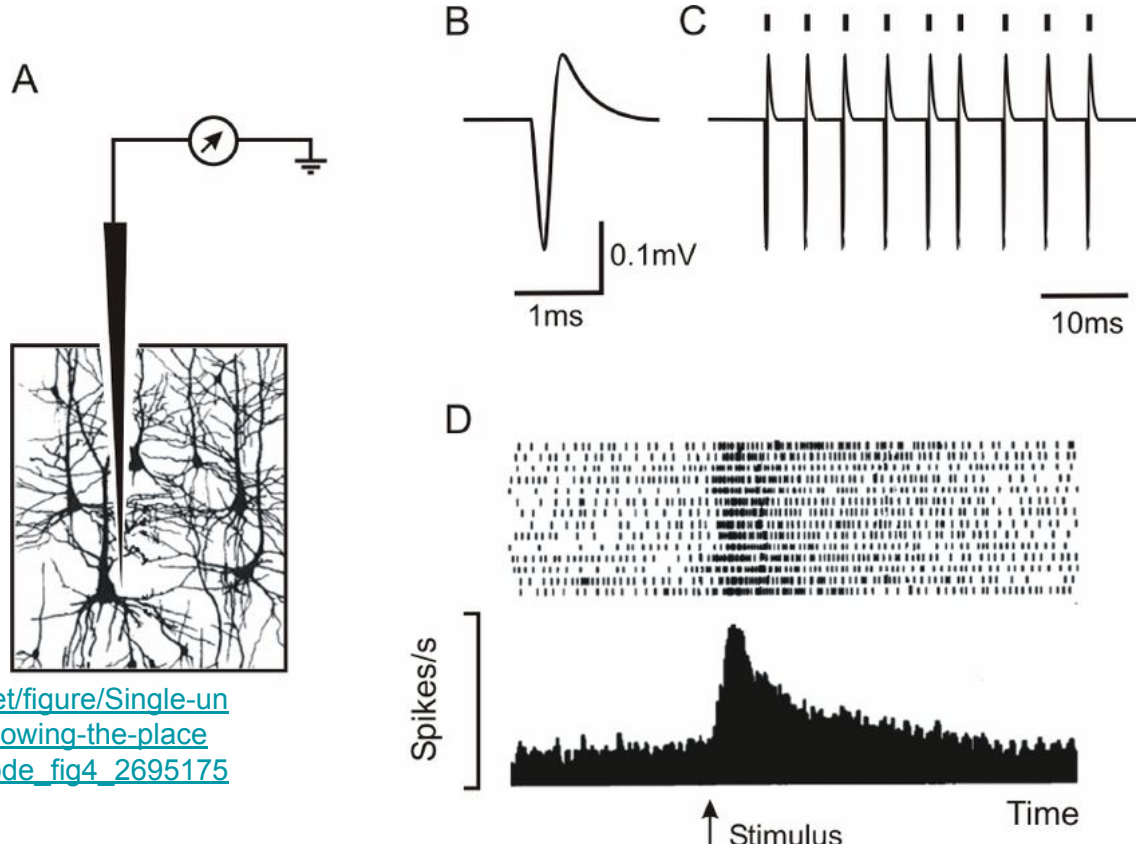
Specifics of images



Spooky story

Or Hubel and Wiesel's Nobel Prize in Medicine

God gave people a needle...



https://www.researchgate.net/figure/Single-unit-recording-A-Schematic-showing-the-place-ment-of-the-tip-of-an-electrode_fig4_269517554

That's what “neural networks” are supposedly trying to replicate.

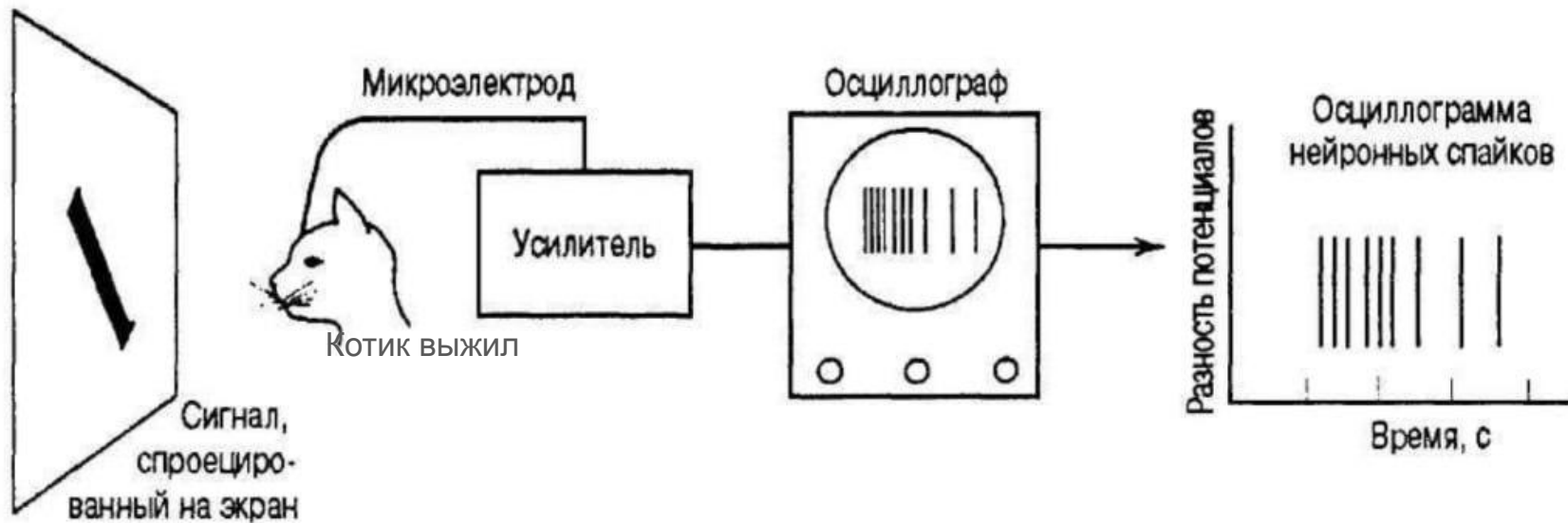
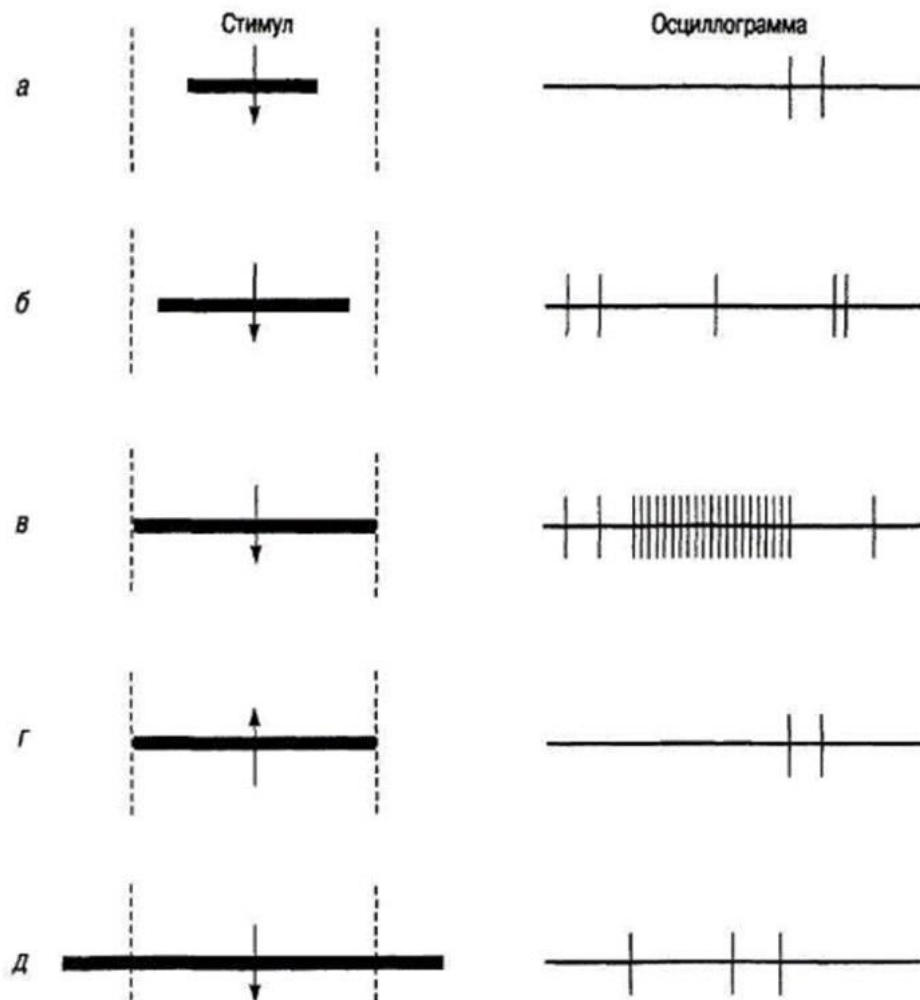
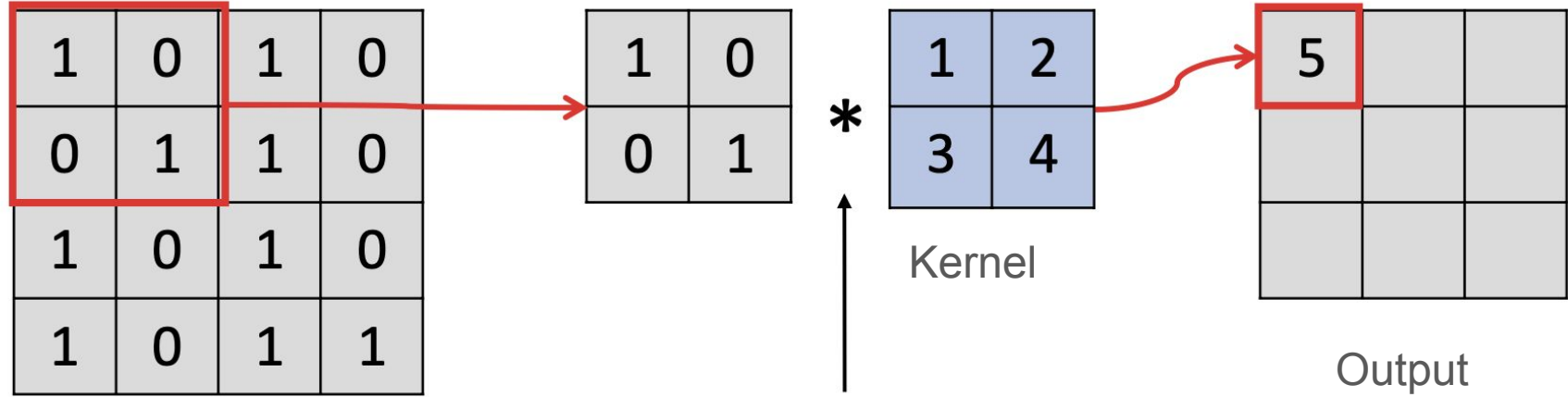


Рис. 3.26. Оборудование для экспериментального обнаружения и изучения рецептивных полей

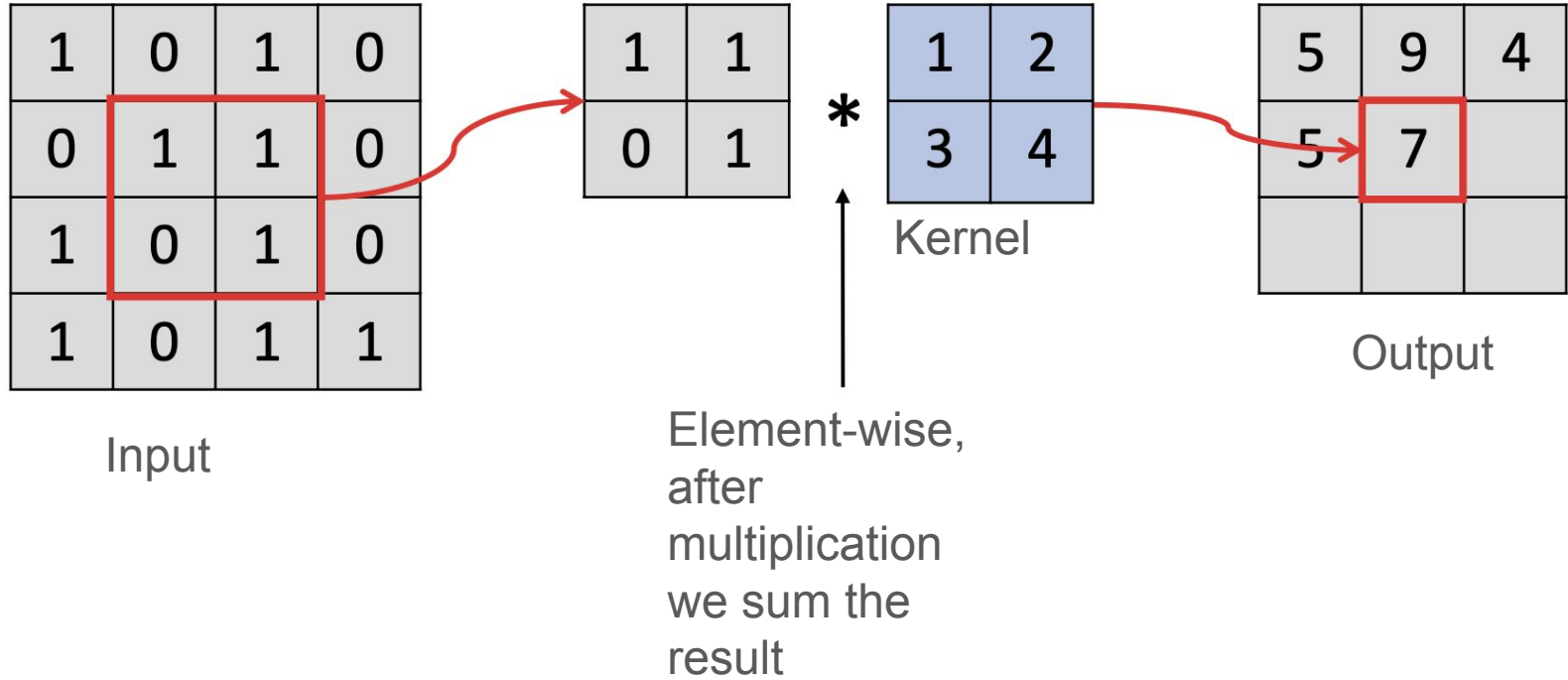


Let's make something "similar"

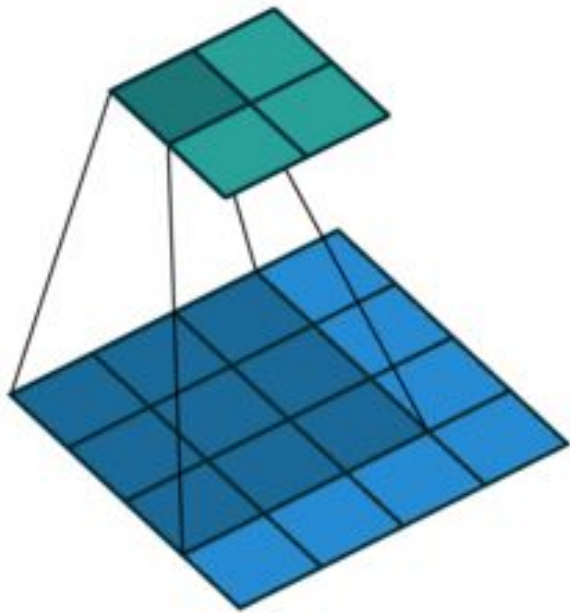


Element-wise,
after
multiplication
we sum the
result

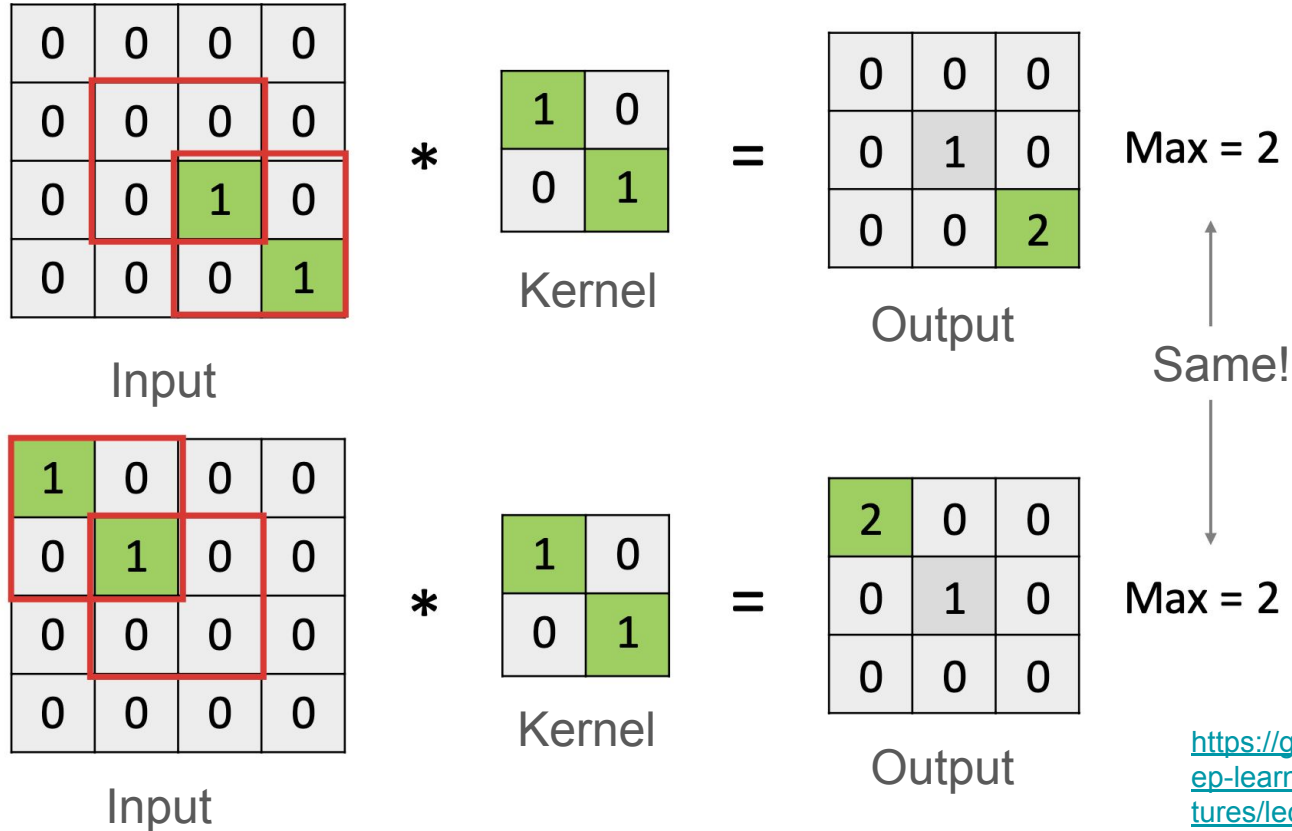
Do the same operation again and again



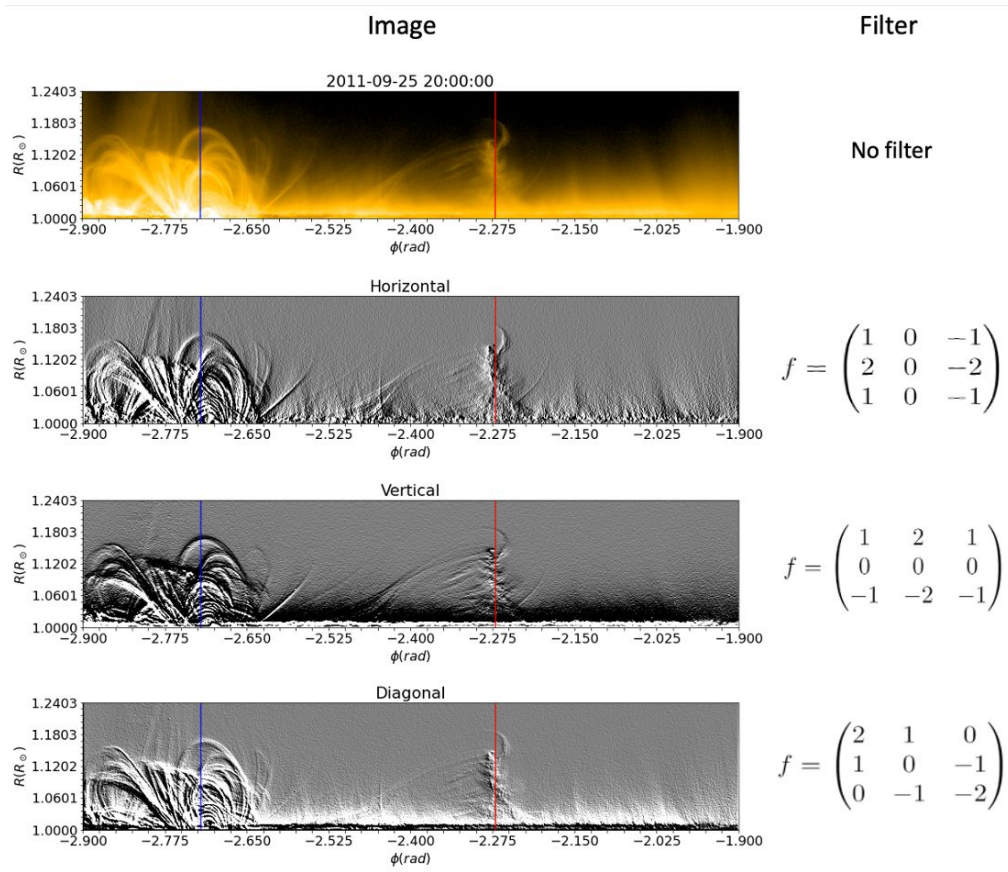
Conv2d



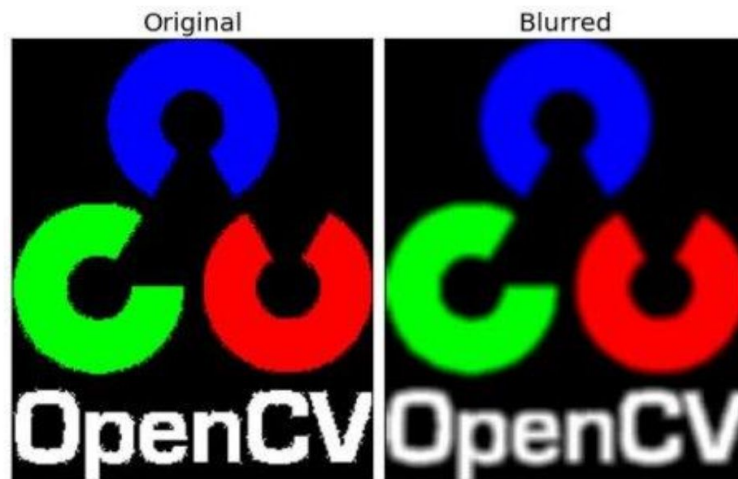
The maximum of the convolution is invariant to shifts



You just reinvented the wheel



OpenCV



$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Do it once

$$\text{Im}^{out}(x, y) = \sum_{i=-d}^d \sum_{j=-d}^d (K(i, j) \text{Im}^{in}(x + i, y + j) + b)$$

Do it twice (RedGreenBlue)

$$\text{Im}^{out}(x, y) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C (K(i, j, c) \text{Im}^{in}(x + i, y + j, c) + b)$$

We want to search for many patterns

$$\text{Im}^{out}(x, y, t) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C (K_t(i, j, c) \text{Im}^{in}(x + i, y + j, c) + b_t)$$

How many parameters are there?

In formula terms:

$$\text{Im}^{out}(x, y, t) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C (K_t(i, j, c) \text{Im}^{in}(x + i, y + j, c) + b_t)$$

$$((2d + 1)^2 * C + 1) * T$$

In PyTorch: K - kernel size, C - channels

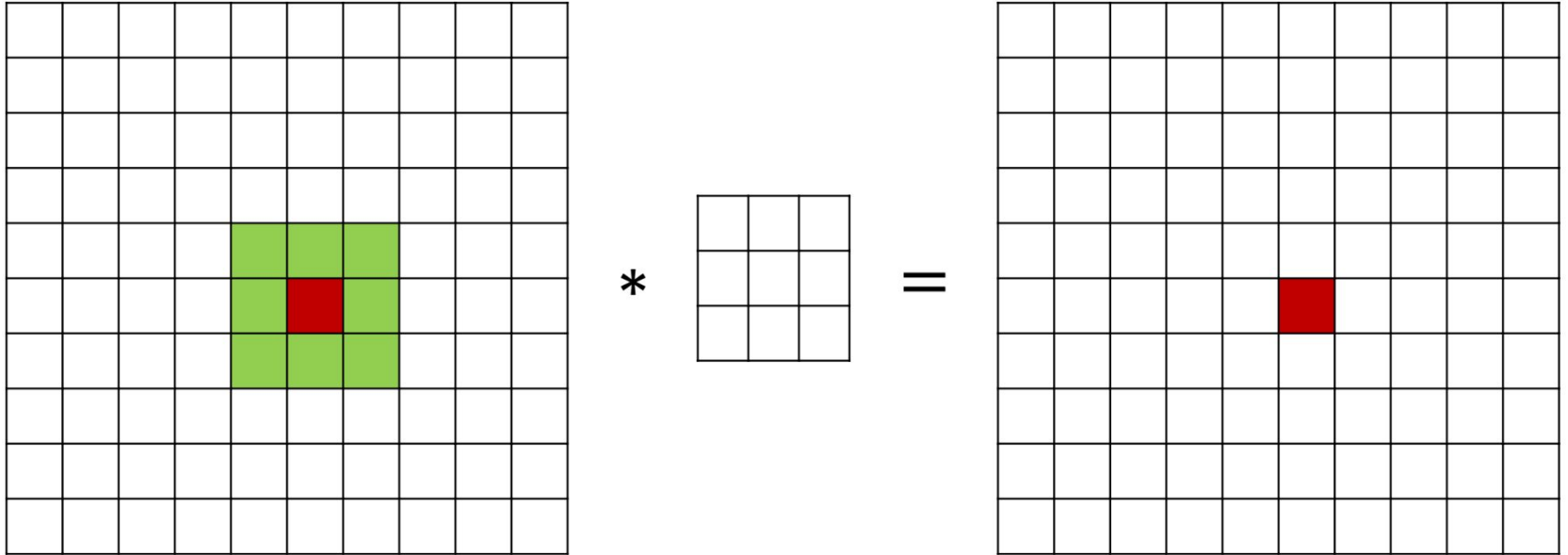
$$K \cdot K \cdot C_{in} \cdot C_{out} + C_{out}$$

Where is the catch?

H, W of the output tensor

$$H \cdot W \cdot C_{out} \cdot (K \cdot K \cdot C_{in})$$

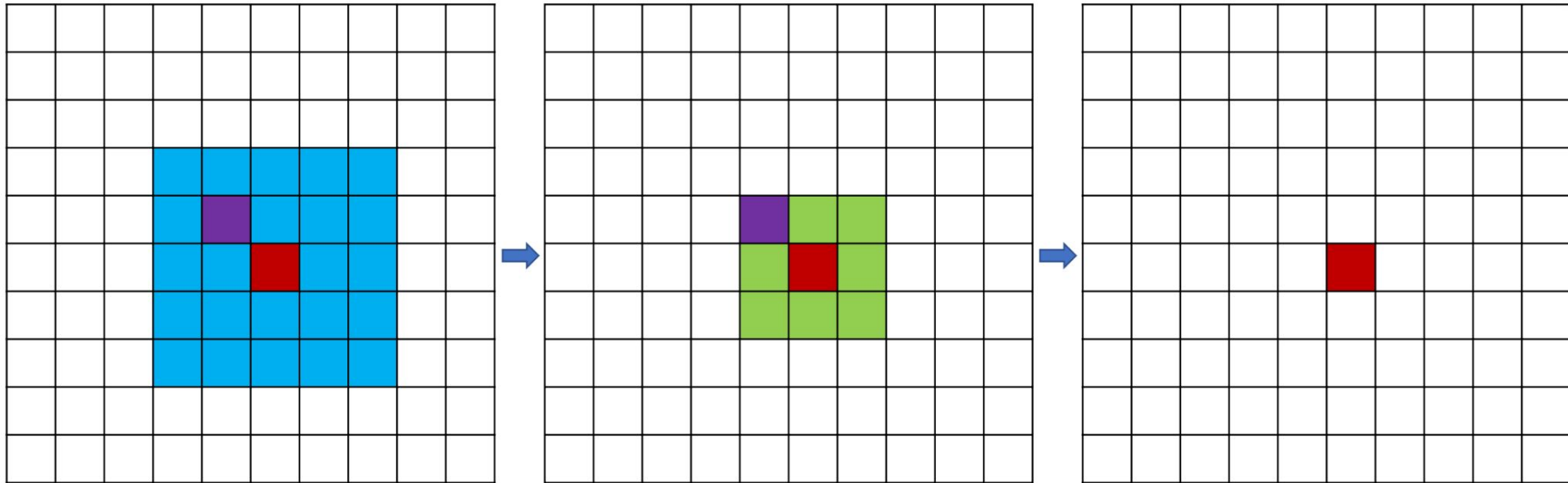
Receptive field (your eye is not the same)



Receptive field : 3 x 3

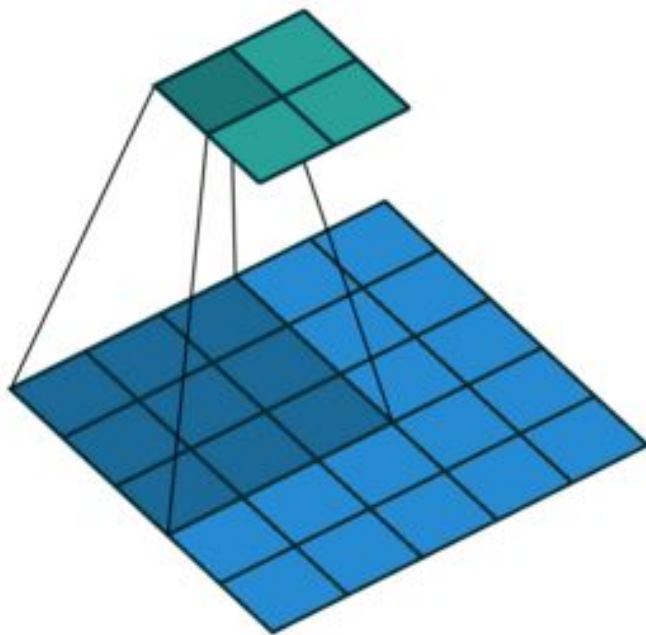
<https://github.com/hse-ds/iad-dep-learning/blob/master/2024/lectures/lecture02-convnets.pdf>

Receptive field

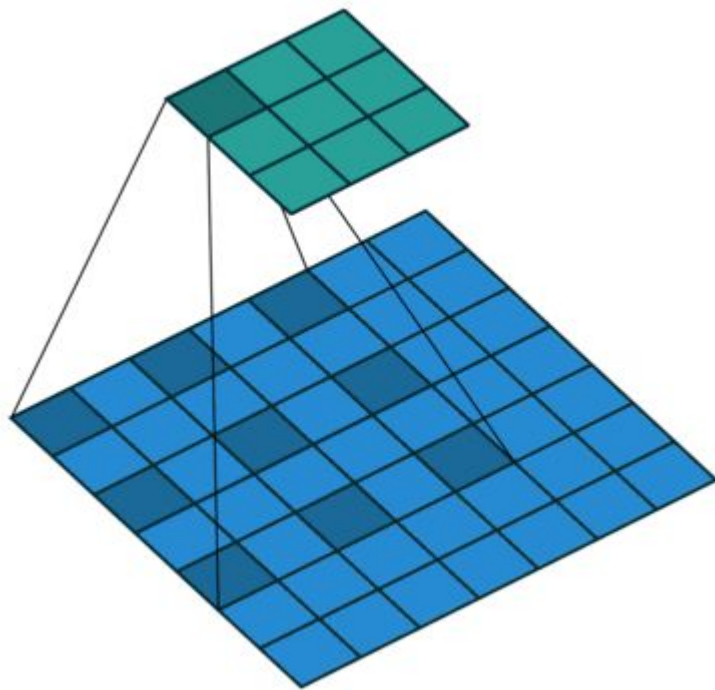


Receptive field : 5 x 5

Stride



Dilation



Pooling (max, etc.)

1	0	2	1	0	0
0	1	3	2	1	2



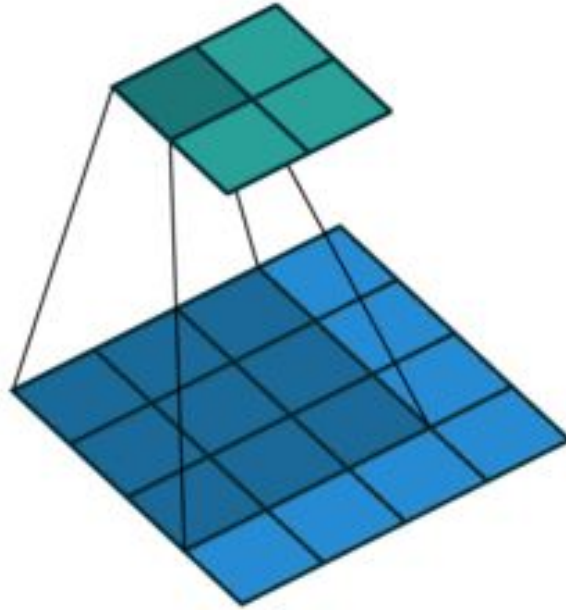
1	3	2

What's the benefit of all this?

We win in the receptive field size (and it seems as if we want the field of perception to be the size of the picture in the last layers)

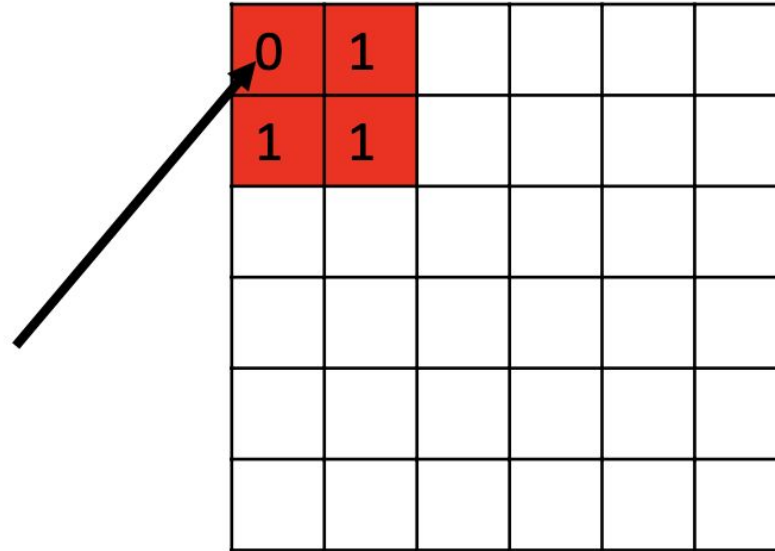
We reduce the size of the image, the number of layers, and the parameter count

But what if edges are important?



Let's see an example

This pattern will
be lost right
now!

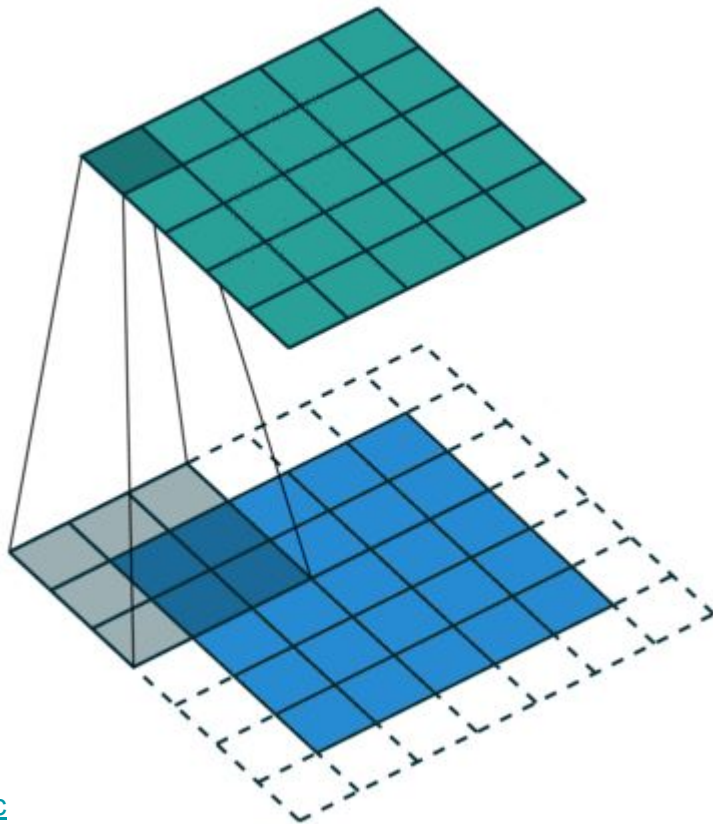


0	1				
1	1				

*

0	0	1
0	0	1
1	1	1

Padding



Zero padding

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

*

=

Reflection

3	6	6	7	8					
8	7	1	2	3					
2	1	1	2	3	4	5	6		
7	6	6	7	8	9	8	7		
2	1	1	2	3					

*

=

Replication

1	1	1	2	3					
1	1	1	2	3					
1	1	1	2	3	4	5	6		
6	6	6	7	8	9	8	7		
1	1	1	2	3					

*

=

How to calculate the size of the output “image”?

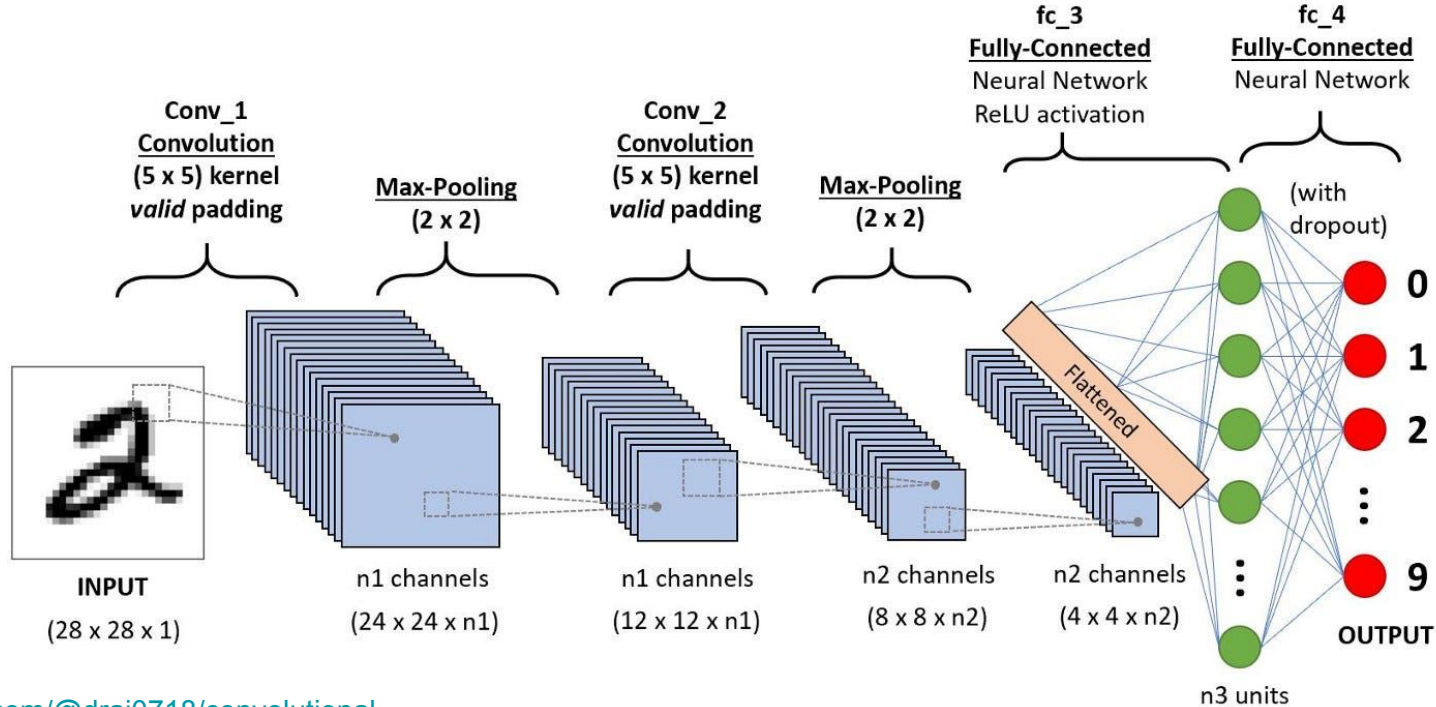
P.S.: Pooling also changes it in this way, just use the formula below :)

$$O = \frac{W - F + 2P}{S} + 1$$

W — input width, F — kernel size,

P — padding, S — stride

Typical CV architecture N years ago



Flatten



Want to learn more?

<https://cs231n.github.io/>